Adrien Deü - Corentin Dupaigne - Nhat Linh Nguyen - Onur Genc

Campagne de Test Complète

Restful-Booker Platform

Dossier 1 : Documentation QA

Stratégie & Plan de Test

Sommaire

PARTIE 1 : STRATÉGIE DE TEST **3**
1. Objectifs de test 3
2. Périmètre (Scope) 3
   2.1 Éléments inclus (IN-SCOPE) 3
   2.2 Éléments exclus (OUT-OF-SCOPE) 3
3. Niveaux et types de tests 4
4. Environnements et outils 4
5. Gestion des anomalies (Defect Management) 5
6. Analyse des risques majeurs 5
7. Focus stratégique : répartition tests manuels / automatisés 6

PARTIE 2 : PLAN DE TEST **7**
1. Rôles et responsabilités 7
2. Calendrier et planning 7
3. Critères d’entrée et de sortie 8
4. Livrables de test 8

PARTIE 1 : STRATÉGIE DE TEST

1. Objectifs de test

L'objectif global de cette campagne de test est de valider la qualité et la fiabilité de la plateforme Restful-Booker avant sa mise en production. Il s'agit d'une application web de réservation de chambres d'hôtel, dotée d'une architecture Microservices, accessible à l'adresse https://automationintesting.online/.

Les objectifs spécifiques sont les suivants :

- Vérifier le bon fonctionnement des fonctionnalités critiques : réservation de chambres, gestion des messages de contact, administration du back-office (création/modification/suppression de chambres).
- Valider la stabilité et la cohérence de l'API REST sous-jacente (codes HTTP, structure JSON, intégrité des données).
- S'assurer de la sécurité de l'authentification administrateur et de la protection des données.
- Détecter les anomalies visuelles et fonctionnelles sur l'interface utilisateur (IHM).
- Protéger l'image de la marque en garantissant une expérience utilisateur fluide et sans bugs bloquants.

2. Périmètre (Scope)

2.1 Éléments inclus (IN-SCOPE)

- Parcours de réservation complet côté utilisateur (sélection de dates, remplissage du formulaire, confirmation).
- Formulaire de contact (envoi de messages depuis la page d'accueil).
- Back-office administrateur : authentification, gestion des chambres (CRUD), consultation des réservations et des messages.
- API REST : endpoints de booking (/booking/), rooms (/room/), auth (/auth/), message (/message/), branding (/branding/).
- Validation des codes de retour HTTP (2xx, 4xx) et de la structure des réponses JSON.
- Vérification de l'IHM : affichage, messages d'erreur, navigation.

2.2 Éléments exclus (OUT-OF-SCOPE)

- Tests de performance et de charge (non demandés dans le cadre du projet).
- Tests d'accessibilité (WCAG).
- Tests de compatibilité multi-navigateurs (les tests seront exécutés sur Chrome uniquement).
- Tests de sécurité avancés (injection SQL, XSS) au-delà de la vérification basique de l'authentification.
- Fonctionnalités de rapport/export qui ne sont pas présentes dans l'application.

3. Niveaux et types de tests

Les types de tests mis en place couvrent plusieurs niveaux complémentaires :

Tests fonctionnels (boîte noire) :

Ces tests vérifient que le comportement de l'application correspond aux résultats attendus sans se préoccuper du code interne. Ils couvrent le parcours utilisateur de bout en bout (réservation, contact) ainsi que les scénarios d'erreur (champs manquants, dates invalides).

Tests d'API :

Ces tests valident la logique métier du back-end de manière isolée. Ils portent sur les codes de retour HTTP, les temps de réponse, la structure du schéma JSON et l'exactitude des valeurs retournées. Un chainage dynamique des requêtes est utilisé pour modéliser des flux complets (génération du token admin, création, lecture, modification, suppression d'une réservation).

Tests d'IHM (Interface Homme-Machine) :

Automatisés via Selenium WebDriver en Python, ces tests couvrent les parcours critiques sur l'interface graphique : scénarios nominaux (Happy Path) et scénarios d'erreur (Negative Path).

Tests de non-régression :

L'ensemble des tests automatisés (API et IHM) constitue une suite de non-régression, qui pourra être ré-exécutée après chaque correction de bug pour s'assurer qu'aucune anomalie n'a été introduite.

4. Environnements et outils

L'application testée est hébergée en ligne sur un serveur dédié. Il n'y a pas de serveur local à configurer. L'environnement de test est directement l'instance déployée sur https://automationintesting.online/.

Outil	Utilisation
Postman	Automatisation des tests d'API, chaînage dynamique des requêtes, assertions JavaScript, gestion des variables d'environnement.
Selenium WebDriver (Python)	Automatisation des tests IHM, architecture Page Object Model (POM), attentes explicites (WebDriverWait).
Chrome (navigateur)	Navigateur utilisé pour l'exécution des tests Selenium et l'analyse réseau (DevTools, onglet Network).
Outils de développement (F12)	Rétro-ingénierie de l'API : identification des endpoints, méthodes HTTP, headers et payloads JSON.

5. Gestion des anomalies (Defect Management)

Chaque anomalie détectée sera documentée dans un rapport de bugs dédié avec les informations suivantes : un identifiant unique, un titre descriptif, les étapes de reproduction, le résultat obtenu, le résultat attendu, la sévérité, la priorité, et si possible des captures d'écran ou des logs réseau.

Niveaux de sévérité :

Sévérité	Description
Critique	Fonctionnalité complètement bloquée, aucune solution de contournement (ex : impossible de créer une réservation).
Majeur	Fonctionnalité dégradée avec impact significatif sur l'expérience utilisateur (ex : mauvais code de retour API).
Mineur	Défaut visuel ou comportement inattendu sans impact bloquant (ex : erreur d'affichage dans un champ).
Cosmétique	Problème purement esthétique sans impact fonctionnel (ex : alignement d'un texte).

Les anomalies seront tracées manuellement dans un fichier dédié (rapport d'anomalies) faisant partie des livrables du projet.

6. Analyse des risques majeurs

Plusieurs risques métiers sont inhérents à une plateforme de réservation hôtelière :

Risque	Impact	Action d'atténuation
Échec de la réservation	Perte directe de chiffre d'affaires	Tests fonctionnels approfondis du parcours de réservation (happy path + cas d'erreur)
Faille d'authentification admin	Accès non autorisé au back-office, modification de données	Tests de l'endpoint /auth/ avec identifiants valides et invalides
Incohérence des données API	Réservations incorrectes, perte de confiance client	Validation des schémas JSON et chaînage dynamique des requêtes
Indisponibilité du serveur de test	Blocage de la campagne de test	Découplage des tests front/back pour limiter les dépendances
Régression après correction	Introduction de nouveaux bugs	Suite de tests automatisés ré-exécutable (non-régression)

7. Focus stratégique : répartition tests manuels / automatisés

La répartition retenue est de 30% de tests manuels et 70% de tests automatisés.

Justification :

Les tests automatisés (70%) couvrent les parcours répétitifs et critiques qui doivent être ré-exécutés régulièrement : les flux API complets via Postman (chaînage CRUD, validation des codes de retour et des schémas JSON) et les parcours IHM principaux via Selenium (réservation, formulaire de contact, gestion admin). L'automatisation garantit la répétabilité et permet de constituer une suite de non-régression fiable.

Les tests manuels (30%) sont réservés à l'exploration initiale de l'application (rétro-ingénierie de l'API via les DevTools, découverte des endpoints), à la vérification visuelle de l'IHM (rendu graphique, ergonomie), et au rapport d'anomalies nécessitant une analyse humaine. Ces tests ne justifient pas un investissement en automatisation car ils sont ponctuels et exploratoires.

PARTIE 2 : PLAN DE TEST

1. Rôles et responsabilités

L'équipe est composée de 4 membres, chacun assurant un rôle défini :

Membre	Rôle	Responsabilités
Adrien	Lead QA / Rédacteur	Rédaction de la stratégie et du plan de test, coordination générale, préparation de la soutenance.
Onur Genc	Automaticien API	Conception et exécution de la collection Postman, chaînage dynamique, scripts d'assertions.
Corentin	Automaticien IHM	Développement des scripts Selenium (POM), couverture des Happy Path et Negative Path.
Nhat-Linh	Testeur / Rapporteur	Conception des cas de test (Excel), exécution des tests manuels, rédaction du rapport d'anomalies.

2. Calendrier et planning

Le projet se déroule sur environ 5 jours avant la soutenance. Voici le planning prévisionnel :

Phase	Activités
Jours 1	Exploration de l'application, rétro-ingénierie de l'API (DevTools), identification des endpoints et des flux métiers.
Jours 2	Rédaction de la stratégie de test, du plan de test et conception des cas de test (fichier Excel).
Jours 3	Développement des tests automatisés : collection Postman (API) et scripts Selenium (IHM).
Jours 4	Exécution des tests, identification et documentation des anomalies (rapport de bugs).
Jours 5	Consolidation des livrables, relecture, préparation de la soutenance et de la démo en direct.

3. Critères d’entrée et de sortie

Critères d’entrée (pour démarrer la campagne de test) :

- L’application est accessible et fonctionnelle sur https://automationintesting.online/.
- Les outils sont installés et configurés (Postman, Python, Selenium, Chrome).
- L’exploration initiale et la rétro-ingénierie de l’API sont terminées.
- Les cas de test sont conçus et validés dans le fichier Excel.

Critères de sortie (pour considérer la campagne terminée) :

- 100% des cas de tests critiques ont été exécutés.
- Aucun bug de sévérité critique n’est en statut “open” sans documentation.
- Tous les tests automatisés (Postman et Selenium) s’exécutent sans erreur technique.
- Les livrables sont complets : documentation QA, collection Postman, scripts Selenium, rapport d’anomalies.

4. Livrables de test

La campagne produira les livrables suivants, organisés en trois dossiers :

Dossier 1 – Documentation QA :

- Stratégie de test (ce document).
- Plan de test (ce document).
- Fichier Excel des cas de test (avec colonnes : ID, Titre, Pré-requis, Étapes, Résultat attendu, Résultat obtenu, Statut, Automatisé).
- Rapport d’anomalies (minimum 3 bugs majeurs/critiques documentés).

Dossier 2 – Automatisation API :

- Collection Postman (export JSON) avec flux complets et assertions.
- Environnement Postman (export JSON) avec les variables d’environnement.

Dossier 3 – Automatisation IHM :

- Code source Python/Selenium (architecture POM).
- Fichier README.md avec instructions de configuration et d’exécution.